claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_dd531439-5e0\agent-a6a560488f72921ae.jsonl`
- SHA-256: `ea59b502d8733e4e9f683a1fa0ca0054fd22c6325e4795d7e542928eeadf4fb9`
- Source modified: `2026-07-08T06:37:31+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`

Transcript

1. user (2026-07-08T06:36:06.854Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).
To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::_md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.
A prior reviewer raised this finding on the DESIGN-DOC ACCURACY (docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md) dimension:
  SEVERITY: nit
  FILE: docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:23
  SUMMARY: orchestration.py line anchors are stale in the committed tree because this same PR shifts those lines.
  CLAIMED FAILURE: §4.1 (line 23) cites `backend/orchestration.py:748` (acquire_local_input) and `:751` (re-hash compute_video_id); §10 (line 218) cites `:741-763` for '(re-hash)'. Those were the pre-diff (HEAD) positions, but this PR's own orchestration.py edit moves them: in the committed worktree acquire_local_input is at :771 and compute_video_id is at :798, and :741-763 now spans the docstring/config-resolution, not the re-hash. Anchors land on the wrong lines on the merged branch.
  PROPOSED FIX: Update to :771 (acquire_local_input) / :798 (re-hash), or drop the line numbers and keep the function-name anchors only.

Adversarially VERIFY it against the real code/doc in C:/Users/avidu/AppData/Local/Temp/claude/wt524-969. Default to REFUTED unless you can construct the concrete failing input/state and confirm it in the actual code. A finding is CONFIRMED only if it is a real defect a maintainer must fix (not a style preference, not already-handled, not a misread). State the verdict and the evidence (file:line you checked).

2. assistant (2026-07-08T06:36:10.424Z)

I'll verify this finding against the actual code and doc in the worktree.

3. user (2026-07-08T06:36:22.413Z)

1. Problem & goal

Why now. Tracing the live CUJ (`runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9 `[verified]`) produced two findings:

1. **Data is moved 3× per run** `[verified]`: (a) upload streams straight to `gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix, `backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane re-downloads the whole clip to scratch to validate ? `_execute_processing` ? `storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a **redundant re-hash** `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect (`[worker] pulled ?`). The unlogged **~30 s pre-validation gap** is finding (b).
2. **The control-plane is the scaling wall** `[verified]`: validation (183 s) **and** stitch (~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-plane, serialized by `_LOCAL_COMPUTE_LANE` ? ~16 min of heavy CPU per run ? the API saturates past ~5 concurrent users.

The concrete outcome we want. The L4 worker becomes the primary compute for the whole pipeline; the control-plane shrinks to a **thin router** (auth · dispatch · DB · SPA + job status · signed-URL download) that does **no heavy decode/encode** and, ideally, **never pulls the full file**. Every move must be **reversible by config** with a **bucket-mediated fallback**, and land as small PRs on top of `origin/master`.

The hard question this doc must resolve (compute topology). "Upload directly to the L4" needs an **inbound endpoint on the GPU box**, but the current L4 is a Cloud Run **Job** (batch, no inbound HTTP ? `backend/compute/cloud_run.py`, dispatched per-request, `[verified]`). So the epic forces a choice between **A. Cloud Run Service-with-GPU**, **B. GCE L4 VM**, and **C. a hybrid** presign/redirect. §4 answers it with numbers.

Read to get current: [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md) ? the `phase_placement` knob this sequences on, [README.md](./README.md) §2 decisions **D4/D7/D16** (the ratified constraints that bound the options), and the runlog §9 (the evidence).
----- LINE 218 area -----
- [x] Phased migration laid out on #521's `phase_placement`, each step a config flip with a bucket-mediated fallback (§5). `[this PR]`
- [x] Win #2 landed (config-guarded, reversible, tests green). `[this PR]`
- [ ] Win #1 confirmed delivered by #521 (not duplicated). `[cross-PR]`
- [ ] Live CUJ confirms the gap is gone + movement is 2× (P3, owner-side). `[gated]`
- [ ] Owner signs off on L5 (reject upload-onto-L4) ? flip Status ? IN PROGRESS/DONE.

10. References

Internal code anchors `[verified]`: `backend/api/uploads.py` (streamed md5), `backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) / `_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`), `backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521 `resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-validation`).
Internal docs: [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md), [README.md](./README.md) (ADR-014, D4/D7/D16), `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9 (evidence).
External `[verified]` rates: GCP Cloud Billing Catalog API (`cloudbilling.googleapis.com`, asia-southeast1, USD, 2026-07-08). **`[researched]`:** same-region GCS?Cloud Run egress treatment (not live-fetchable here).

Changelog
- `2026-07-08` ? initial draft: topology A/B/C resolved (recommend A? = co-located Jobs + bucket upload; reject persistent-GPU + upload-onto-L4 with a cost model); durability + phased migration on #521's `phase_placement`; win #2 landed alongside.

4. user (2026-07-08T06:36:22.900Z)

16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
244:    ``compute_video_id`` produces (`backend/utils/hashing.py`). Guards the #524 win-#2 trust path:
736:    ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
771:        local_video = storage.acquire_local_input(
776:        # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-time
787:        # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
798:            video_id = _main.compute_video_id(local_video)
1206:   local_src = storage.acquire_local_input(

5. assistant (2026-07-08T06:36:27.972Z)

Let me get exact line numbers in the doc for these references.

6. user (2026-07-08T06:36:34.846Z)

15:The cheaper, more durable, already-ratified path is the opposite: **keep the upload streaming to the cheap bucket** (the #480 design ? the durable copy exists *before* the L4 ever starts), and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ? which is exactly what **#521's `phase_placement` already wires as a config flip**. The residual "~30 s pre-validation gap" is then killed not by moving the *upload* but by the control-plane **never downloading the file at all** (it delegates validation to the L4 per #521, and trusts the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops from 3× to the theoretical floor: **1 client upload + 1 LAN-fast intra-region pull**, with **zero idle-GPU cost** and **zero exposure window**.
17:**Actionable now (this PR):** the design doc + **win #2** (drop the redundant control-plane re-hash, config-guarded, reversible). **Win #1** (validation-on-L4) is **delivered by #521** ? this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is **rejected at current volume** on a quantified cost basis and left as an owner-gated future ADR.
23:1. **Data is moved 3× per run** `[verified]`: (a) upload streams straight to `gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix, `backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane re-downloads the whole clip to scratch to validate ? `_execute_processing` ? `storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a **redundant re-hash** `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect (`[worker] pulled ?`). The unlogged **~30 s pre-validation gap** is finding (b).
38:| L3 | **Kill the ~30 s gap by *not downloading to the control-plane*, not by uploading onto the L4.** Compose #521's validation-delegation (CP skips the validator suite) with win #2 (CP trusts the submit-time md5 ? no re-hash). | this doc `[design]` | The CP stops pulling the full file when validation is delegated; movement ? 3×?2×. |
41:| L6 | **Win #2 (drop the re-hash) is config-guarded + reversible** (`deployment.trust_submit_video_id`, default on) and produces a **byte-identical `video_id`** (the GCS `md5Hash` over the same bytes = `compute_video_id`). | win #2 `[verified]` | Zero-regression; one-flag rollback. |
68:| Accept ? validate start | ~30 s | control-plane | no | **the gap** = CP re-download to scratch (`acquire_local_input` + re-hash) |
147:| **~~Full-file download + re-hash~~** | ? **eliminated** (delegate validation + trust md5, win #2) | this PR |
166:| **P2a** | **Drop the CP re-hash** (trust submit-time md5) | `deployment.trust_submit_video_id` (default on) | re-hash on the drain | **this PR (win #2)** |
178:- **Win #2 changes nothing when no trusted id is present** ? local/appliance uploads and non-GCS direct-URI submits (`submit_time_video_id` ? `None`) still re-hash. The win applies to the GCS cloud-serving path (the CUJ case).

```
190:| P1 | **Win #2** ? drop the CP re-hash; trust submit-time md5;
`deployment.trust_submit_video_id` (default on), reversible | ? | No | ? (this PR) | #524 |
218:**Internal code anchors `[verified]`:** `backend/api/uploads.py` (streamed md5),
`backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) /
`_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the
md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`),
`backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521
`resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-
validation`).
```

7. user (2026-07-08T06:36:35.014Z)

```
    job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
    ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
    ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-hashing the
    multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes as
before.
    It is a keyword-only, internal channel ? NOT a client-supplied request field (a caller's
    ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative id).

    Raises on unexpected internal errors ? the caller records those as a job failure
    """
    import backend.main as _main  # call-time seam: main's globals/model stay patchable

    # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
    # shift positional args ? req arrives in position 0 (config), progress in position 1 (db).
    if config is not None and not isinstance(config, AppConfig):
        # Old signature: _execute_processing(req, progress=None)
        if req is None and db is not None and isinstance(db, _main.ProcessRequest):
            req = db
            db = None
        elif req is None and isinstance(config, _main.ProcessRequest):
            req = config
            config = None
    # Resolve config/db from module globals when not passed explicitly
    if config is None:
        config = _main.global_config
    if db is None:
        db = _main.db_instance

    notify = progress or (lambda stage: None)

    notify("hashing")
    # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged (identity ?
    # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
    # req.video_path (the source URI) is kept for the DB record + report; only the file-reading
    # consumers (hash / validators / segmenter) use the resolved local path.
    local_video = ""
    try:
        local_video = storage.acquire_local_input(
            req.video_path, getattr(config, "storage", None)
        )
        # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing the whole
        # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
        # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-
time
        # cache probe trusts, so reusing it here is consistent, not a new trust assumption.
Gated by
        # deployment.trust_submit_video_id (default on) and only honoured when the id is well-
formed
        # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The fetch
above
        # is unchanged; only the redundant hash is skipped.
        trust_submitted = bool(
            getattr(
```

```
                getattr(config, "deployment", None), "trust_submit_video_id", True
            )
        )
        # STRICT: trust only an id in the EXACT form the drain would have produced (32-char
        # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
        # differently-cased or malformed id is a signal something is off, so re-hash rather than
        # silently coerce it into the DB key.
        candidate = trusted_video_id or ""
        if trust_submitted and _looks_like_video_id(candidate):
            video_id = candidate
            logger.info(
                "[API] trusting submit-time video_id=%s ? skipped the drain re-hash (#524).",
                video_id,
            )
        else:
            video_id = _main.compute_video_id(local_video)
    except Exception:
        storage.cleanup_acquired_local_input(
            req.video_path, local_video, getattr(config, "storage", None)
        )
        raise
    try:
        existing_video = db.get_video(video_id)
```

8. assistant (2026-07-08T06:36:50.109Z)

Let me verify the exact content at the cited line ranges and confirm the drift was introduced by this PR.

9. user (2026-07-08T06:36:53.597Z)

```
=== lines 741-763 (doc §10 cites as re-hash) ===

    Raises on unexpected internal errors ? the caller records those as a job failure
    """
    import backend.main as _main  # call-time seam: main's globals/model stay patchable

    # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
    # shift positional args ? req arrives in position 0 (config), progress in position 1 (db).
    if config is not None and not isinstance(config, AppConfig):
        # Old signature: _execute_processing(req, progress=None)
        if req is None and db is not None and isinstance(db, _main.ProcessRequest):
            req = db
            db = None
        elif req is None and isinstance(config, _main.ProcessRequest):
            req = config
            config = None
    # Resolve config/db from module globals when not passed explicitly
    if config is None:
        config = _main.global_config
    if db is None:
        db = _main.db_instance

    notify = progress or (lambda stage: None)
```

10. user (2026-07-08T06:36:55.130Z)

```
=== HEAD positions of acquire_local_input / compute_video_id ===
16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
748:        local_video = storage.acquire_local_input(
751:        video_id = _main.compute_video_id(local_video)
1159:         local_src = storage.acquire_local_input(
```

11. assistant (2026-07-08T06:37:22.216Z)

Verified. The finding is accurate.

HEAD (pre-diff) positions (`git show HEAD:backend/orchestration.py`):
- `acquire_local_input` in `_execute_processing` = line **748**
- `compute_video_id` re-hash = line **751**

Committed worktree (HEAD + this PR's uncommitted diff) ? the PR's own orchestration.py edit shifted them down:
- `acquire_local_input` = line **771**
- `compute_video_id` re-hash = line **798**
- Lines **741?763** now span the docstring tail, `import backend.main`, the backward-compat positional-shim, config/db resolution, and `notify = progress ?` ? **not** the re-hash (which sits at 798, outside that range).

Doc citations that are now stale:
- `docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:23` (§4.1) cites `backend/orchestration.py:748` (acquire_local_input) and `:751` (re-hash) ? both off (actual 771 / 798).
- `docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:218` (§10) cites `:741-763` as "(re-hash)" ? that range is the docstring/shim/config-resolution block, not the re-hash.

Both anchors carry the `[verified]` tag, so the stale numbers directly contradict that claim. This is a genuine, concrete inaccuracy introduced by this PR ? CONFIRMED ? but nit-severity (doc line-anchors, no runtime/behavioral impact; function-name anchors in the same sentences remain correct).

12. user (2026-07-08T06:37:31.593Z)

Structured output provided successfully